

ADaRES: Adaptive Data Reallocation and Epoch Scheduling in Federated Learning Under Network and Resource Constraints

Skanda M Bhairava, Gururaj H L, *Senior Member, IEEE*

Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India

Abstract—Federated Learning (FL) efficacy in practical edge and IoT environments is often hindered by client unavailability and non-IID data. This paper addresses the essential problem of client unavailability, in which data from entire classes may become inaccessible. We present ADaRES, a framework for Adaptive Data Reallocation and Epoch Scheduling. We provide an overview of the ADaRES system, which detects clients with limited resource that are reachable via network, redistributes their data to stable peers, and adjusts local epochs to evenly the new computational load. We assess ADaRES within a challenging simulation setup based on Perlin noise, which simulates realistic, time dependent client availability. Our experiments reveal that ADaRES achieves a final accuracy (76.9%) comparable to that of an ideal system with constant availability, while a conventional FL model in the same unstable environment fails to converge (49.1%). We conclude that ADaRES serves as a computationally-efficient and resilient solution to the challenge of client unavailability in FL.

Index Terms—Federated Learning, Edge Computing, Data Reallocation, Resource-Aware, Non-IID Data, Client Availability

I. INTRODUCTION

FEDERATED LEARNING (FL) facilitates collaborative model training on decentralized data, providing significant privacy benefits [1]. However, the conventional FL algorithm encounters difficulties in practical applications [2], especially in Internet of Things (IoT) and mobile edge environments [3, 4]. These environments present two significant challenges: severe data heterogeneity (non-IID) [5] and inherent client unreliability [6]. Recent work highlights an important distinction: the issue is often not just “stragglers” (slow clients), but complete client unavailability, where clients may not participate at all, leading to critical data loss [7].

In many realistic deployments, client devices operate within a trusted edge domain where peer devices, but not the central server, are allowed to store encrypted replicas of each other’s data. This privacy model appears in controlled environments such as enterprise IoT, industrial clusters, and cooperative home networks. Our work assumes this class of deployments, where limited peer-to-peer data movement is permissible while the core FL objective of avoiding server-side data exposure is preserved.

Our work focuses on situations where these challenges are linked: a non-IID distribution where most instances of specific classes are hosted on clients that are often unavailable due to limited resources (e.g., low battery, high CPU usage). An

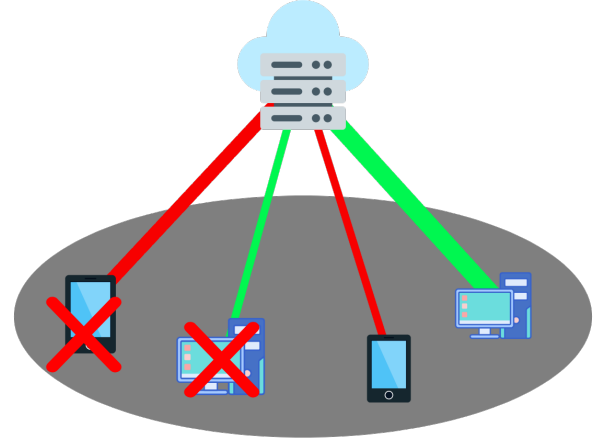


Fig. 1: High-level overview of the System architecture.

overview is shown in Fig. 1. In such a scenario, the global model will not succeed in learning these classes, regardless of the optimization strategy used.

To address this, we propose ADaRES, an Adaptive Data Relocation and Epoch Scheduling framework. ADaRES oversees the resource and network statuses of clients. When a client experiences resource constraints but sufficient network bandwidth, ADaRES actively reallocates that client’s data to reliable, available peers.

Our key contributions are:

- FL framework ADaRES, that addresses extreme data imbalance by intelligently redistributing data from clients with limited resources. This system detects resource-poor but network-accessible clients and shifts their data to robust peers, ensuring that no data modalities are permanently lost from the training pool.
- An adaptive local epoch scheduling mechanism that dynamically adjusts the computational burden on hosting clients. This feature is important for system-wide load balancing, ensuring fairness sustainability in training across diverse devices.
- Experimental validation on CIFAR-10 showing that ADaRES (76.9% accuracy) matches an ideal, always stable and available system, while the ‘Reality’ model suffers (49.1%). ADaRES achieves this with a lower average computational load (3.74 epochs) compared to

the failing model (3.96 epochs).

The structure of the remainder of this paper is as follows: Section II reviews related literature. Section III outlines the design and methodology of the ADaRES system. Section IV explains the experimental setup. Section V discusses our findings. Finally, Section VI concludes the paper.

II. RELATED WORK

Our work is focused on the intersection of major challenge areas in FL- data heterogeneity (non-IID), client unreliability, and data-centric solutions.

A. Non-IID Data and Client Heterogeneity

The issue of non-IID data, where client data distributions differ, is well-recognized [5]. Numerous algorithmic solutions have been proposed. FedProx [8] addresses system heterogeneity by allowing for partial contributions and incorporating a proximal term to stabilize the training process. SCAF-FOLD [9] uses control variates to adjust for client drift, and Ditto [10] introduces a personalization-based approach aimed at enhancing fairness and robustness. Though these methods improve resilience to statistical heterogeneity, they do not fundamentally address the problem of data unavailability when all clients that possess a specific class are offline.

B. Client Unavailability

Client unreliability is a significant characteristic of both large-scale and edge-based FL [2, 6]. Much of existing work centers on slow clients that take longer to return updates. However, as pointed out in [7], complete client unavailability is a separate and critical problem. Our scheme directly targets this problem, not by waiting for clients, but by proactively moving their data.

C. Data-Centric Approaches

Another line of research tackles non-IID data by altering the data itself. This includes data-free knowledge distillation, where a server model is trained using synthetic data [11], or hybrid approaches that utilize dataset distillation to share small, synthetic summaries of data [12]. ADaRES is also a data-centric strategy, but with a difference- rather than generating or distributing synthetic data, it reallocates the original, high-quality data from unavailable clients to available ones.

III. SYSTEM DESIGN AND METHODOLOGY

A. Architecture Overview

Our setup consists of a central server and N clients. Each client i possesses a local dataset D_i alongside two state time-series that fluctuate over time: resource strength $R_i(t)$ and network strength $N_i(t)$, where t indicates the FL round. To emulate realistic environmental variations, these time-series are generated using Perlin noise, which results in temporally correlated, non-repeating variations.

We model a complex non-IID situation where data and reliability are linked. A global parameter Υ (Upsilon) denotes

Algorithm 1 ADaRES: Redistribute Data

Require: t (round), Γ (threshold), α (sim. factor), K_{top} , L_{max} (max load)

```

1:  $C_{down} \leftarrow \{i \mid R_i(t) < \Gamma \text{ and } N_i(t) \geq \Gamma\}$ 
2:  $C_{up} \leftarrow \{j \mid R_j(t) \geq \Gamma \text{ and } N_j(t) \geq \Gamma\}$ 
3: for  $d \in C_{down}$  (not already reassigned) do
4:    $Scores \leftarrow []$ 
5:   for  $h \in C_{up}$  (where  $Load(h) < L_{max}$ ) do
6:     if  $\alpha < 0$  then
7:        $S_h \leftarrow -\alpha \cdot -Sim(d, h) + (1 + \alpha) \cdot R_h(t)$ 
8:     else
9:        $S_h \leftarrow \alpha \cdot Sim(d, h) + (1 - \alpha) \cdot R_h(t)$ 
10:    end if
11:     $Scores.append((h, S_h))$ 
12:  end for
13:   $ChosenHosts \leftarrow TopK(Scores, K_{top})$ 
14:  Split and assign  $Data_d$  to  $ChosenHosts$ 
15:  Record assignment in  $TempAssignments[d]$ 
16: end for
17: for  $h \in C_{up}$  do
18:    $E_h \leftarrow E_{base} - f(Load(h), E_{min})$  {Reduce epochs}
19: end for
```

the division between "good" and "bad" classed. A selection of classes, C_{bad} , representing $(1 - \Upsilon)$ of the total classes, is classified as "bad". Clients with data from C_{bad} are assigned a lower-baseline Perlin noise profile for their resource strength $R_i(t)$. A different, independent Υ -split is used to assign network strength $N_i(t)$ profiles.

A global threshold Γ (Gamma) is used to assess availability.

- Bad Resource: Clients with $R_i(t) < \Gamma$ are unable to participate in local training for round t .
- Bad Network: Clients with $N_i(t) < \Gamma$ can engage in training, but cannot participate in aggregation.

ADaRES focuses on the (Bad Res, Good Net) clients, those capable of uploading their data but unable to train. We also investigate if re-allocating data to clients with similar existing datasets proves more effective, controlled by a similarity factor α .

B. Data and Epoch Distribution Scheme

The ADaRES scheme initiates at the beginning of each FL round, managed by three main algorithms.

Algorithm 1 details the data reallocation process. It identifies clients that are "down" (Bad Resource) but "reachable" (Good Network). For each of these clients, it selects the top K most suitable "host" clients. The "down" client's data is split and sent to these hosts. The local epoch counts E_h for "up" clients h are then reduced based on its new data load L_h , where clients with less data load are assigned lower epochs.

Algorithm 2 manages data recovery. At the start of a round, it checks if any previously "down" clients have recovered their resources. If this is the case, it reverts the temporary assignments, retrieving the data indices from the hosts and restoring them to the original owner. All client epoch counts

Algorithm 2 ADaRES: Recover Reassignments**Require:** t (round), Γ (threshold)

- 1: $C_{\text{recovered}} \leftarrow \{i \mid R_i(t) \geq \Gamma \text{ and } i \in \text{TempAssignments}\}$
- 2: Reset all E_i to E_{base}
- 3: **for** $d \in C_{\text{recovered}}$ **do**
- 4: Retrieve data from hosts listed in $\text{TempAssignments}[d]$
- 5: Restore data to d
- 6: Remove d from TempAssignments
- 7: **end for**

Algorithm 3 ADaRES: Full Training Round**Require:** Rounds , Γ , α

- 1: **for** $t \in \{1, \dots, \text{Rounds}\}$ **do**
- 2: Call Recover_Reassignments(t, Γ)
- 3: Call Redistribute_Data($t, \Gamma, \alpha, \dots$)
- 4: $\text{Updates} \leftarrow []$
- 5: **for** $i \in \{1, \dots, N\}$ (where $R_i(t) \geq \Gamma$) **do**
- 6: $w_i \leftarrow \text{ClientTrain}(w_{\text{global}}, \text{Data}_i, E_i)$
- 7: **if** $N_i(t) \geq \Gamma$ **then**
- 8: $\text{Updates.append}(w_i)$
- 9: **end if**
- 10: **end for**
- 11: $w_{\text{global}} \leftarrow \text{Aggregate}(\text{Updates})$
- 12: **for** $i \in \{1, \dots, N\}$ (where $N_i(t) \geq \Gamma$) **do**
- 13: SendModel(i, w_{global})
- 14: **end for**
- 15: **end for**

are reset to the base value before Algorithm 1 is executed again.

Algorithm 3 outlines the complete ADaRES federated learning round. It incorporates the recovery and redistribution steps before moving on to standard training and aggregation. The important difference is that training only occurs on "up" clients with $R_i(t) \geq \Gamma$, and aggregation is restricted to "reachable" clients with $N_i(t) \geq \Gamma$.

We measure the system's overall computational load through the Average Local Epochs per client per round, $E[\text{Epochs}]$, defined as:

$$E[\text{Epochs}] = \frac{1}{T \cdot N} \sum_{t=1}^T \sum_{i=1}^N E_{i,t} \quad (1)$$

where T is the total rounds, N is the total clients, and $E_{i,t}$ is the number of local epochs for client i at round t ($E_{i,t} = 0$ if the client is down).

IV. EXPERIMENTAL SETUP

We assess the performance of ADaRES using the CIFAR-10 dataset, which is allocated among 30 clients in a heavily sharded non-IID fashion (max 2 classes assigned per client). A simple CNN model is used for classification. All hyperparameters can be found in Table I.

TABLE I: Hyperparameters

Parameter	Value
Dataset	CIFAR-10
Clients	30 (2 classes/client)
FL Rounds	30
Good/Bad Upsilon (Υ)	0.7 (70% good clients/network)
Control Gamma (Γ)	0.5
Max Epochs per Round (E_{base})	8
Min Epochs per Round (E_{min})	5
Learning Rate (lr)	0.01
Batch Size	32
Optimizer	torch.optim.SGD
Loss Function	torch.nn.CrossEntropyLoss
Similarity Metric (α)	Cosine similarity of class histograms

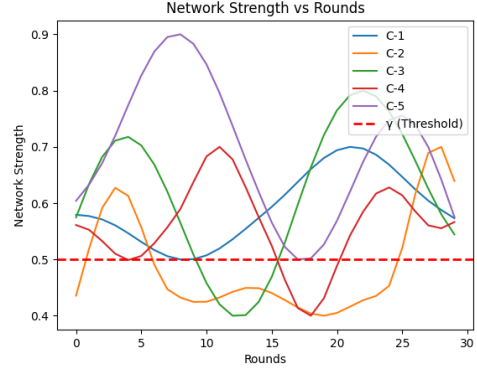


Fig. 2: Simulated Network for the top 5 clients over 30 rounds, generated using Perlin noise. The $\Gamma = 0.5$ cutoff (dashed line) determines client availability.

Fig. 2 displays a sample of the Network Strength time-series generated by Perlin noise for the top five clients. The variable nature and the distinct cutoff line at $\Gamma = 0.5$ highlight the dynamic client availability that our system must handle.

A. Models Compared

We evaluate and compare the performance of the following models:

- ADaRES ($\alpha \in \{-1, -0.5, 0, 0.5, 1\}$): Our proposed model. We test five different values for the similarity factor α , to determine if data similarity influences the convergence of model accuracy, where a high α indicates that data is distributed to other clients that possess similar data.
- Reality: A conventional FedAvg model operating in an unstable environment without data reallocation. Clients conduct 8 local epochs whenever they can run.
- Baseline 8 Epochs: An optimal FedAvg model where all clients consistently remain stable and available. It performs 8 local epochs per client in each round.
- Baseline 10 Epochs: Similar to the previous model, but it performs 10 local epochs.

Training was implemented in PyTorch on an Intel i5 CPU with 16 GB RAM and a GTX 1650 Ti GPU.

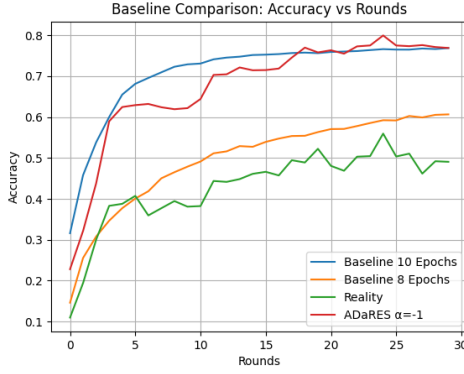


Fig. 3: Model accuracy vs FL rounds. ADaRES ($\alpha=-1$), Baseline 8, Baseline 10, Reality

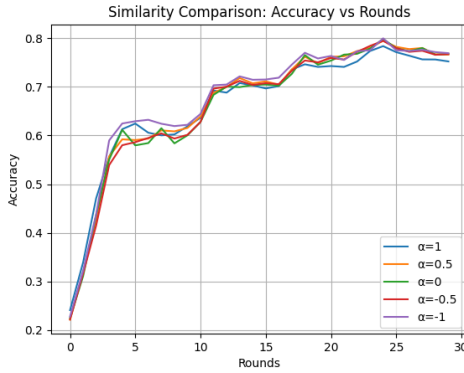


Fig. 4: Comparison of ADaRES performance using different α (similarity) values.

V. RESULTS AND DISCUSSION

A. Accuracy and Convergence

Fig. 3 shows the global model accuracy versus FL rounds. The 'Reality' model struggles to converge, displaying significant fluctuations and ultimately stagnating at an accuracy of 49.1%. This supports our hypothesis that the absence of data due to linked class/resource failures has dire consequences.

In sharp contrast, our ADaRES model (using $\alpha = -1$ for this plot) exhibits excellent stability, achieving a final accuracy of 76.9%. It successfully manages the unstable conditions by ensuring the availability of all data classes. Its performance closely aligns with that of the 'Baseline 10' model (76.8%). This shows that ADaRES effectively compensates for the performance lost to client instability.

Fig. 4 compares the performance of ADaRES where data is distributed to other clients with various similarity scores, i.e. using different α values. We observe that all values produce very similar convergence curves and final accuracies. This suggests that in our highly sharded (2-class) setup, the specific choice of host client is not as important as the reallocation itself. Ensuring that the data is available is the important factor.

Table II presents a summary of the final accuracies. ADaRES significantly outperforms the 'Reality' model by over 27 percentage points.

TABLE II: Final Accuracy at Round 30

Model	Final Accuracy	Improvement vs Reality
Baseline 10	0.7689 (76.9%)	+27.8%
Baseline 8	0.6067 (60.7%)	+11.6%
ADaRES ($\alpha = -1$)	0.7692 (76.9%)	+27.8%
ADaRES ($\alpha = -0.5$)	0.7672 (76.7%)	+27.6%
ADaRES ($\alpha = 0$)	0.7677 (76.8%)	+27.7%
ADaRES ($\alpha = 0.5$)	0.7662 (76.6%)	+27.5%
ADaRES ($\alpha = 1$)	0.7521 (75.2%)	+26.1%
Reality	0.4907 (49.1%)	-

B. Local Epoch Scheduling

The effectiveness of ADaRES can be attributed to its adaptive epoch scheduling. Table III shows the average local epochs per client per round, as defined in Eq. (1).

TABLE III: Average Local Epochs per Client per Round

Model	Avg. Epochs (per client/round)
Baseline 10	10.00
Baseline 8	8.00
Reality	3.96
ADaRES (all α)	3.74

While the 'Reality' model shows a low epoch count (3.96), this is due to clients not participating, leading to unsuccessful training. ADaRES achieves an even lower average epoch count (3.74) while delivering the maximum accuracy of the 10-epoch baseline using only 37.4% of the total computational effort.

C. Disadvantages and Drawbacks

A notable downside of ADaRES is the cost of data transfer. For the model to function effectively, clients with poor resources must have sufficient network bandwidth to upload their complete local dataset. This reallocation cost can be significant and could be a bottleneck in environments with limited bandwidth. ADaRES is not intended for cross-silo or cross-device FL with strict data immobility requirements.

VI. CONCLUSION AND FUTURE WORK

This paper introduced ADaRES, a framework that addresses the essential relationship between data-class distribution and client unavailability in Trusted-Edge environment FL. By dynamically redistributing data from resource-limited yet accessible clients and adaptively scheduling local epochs, ADaRES achieves optimal accuracy even in highly unstable environments.

For future work, we intend to incorporate a more advanced cost model. Rather than a static Γ threshold, an AI-driven model could predict the duration of a client's unavailability. This would enable ADaRES to make more intelligent decisions, minimizing costly data transfers for clients that are expected to recover quickly.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [2] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, B. McMahan *et al.*, "Towards federated learning at scale: System design," *Proceedings of machine learning and systems*, vol. 1, pp. 374–388, 2019.
- [3] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE communications surveys & tutorials*, vol. 22, no. 3, pp. 2031–2063, 2020.
- [4] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for internet of things: A comprehensive survey," *IEEE communications surveys & tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.
- [5] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-iid data," *arXiv preprint arXiv:1806.00582*, 2018.
- [6] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [7] D. Garg, D. Sanyal, M. Lee, A. Tumanov, and A. Gavrilovska, "Client availability in federated learning: It matters!" in *Proceedings of the 5th Workshop on Machine Learning and Systems*, ser. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 114–121. [Online]. Available: <https://doi.org/10.1145/3721146.3721964>
- [8] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine learning and systems*, vol. 2, pp. 429–450, 2020.
- [9] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning*. PMLR, 2020, pp. 5132–5143.
- [10] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *International conference on machine learning*. PMLR, 2021, pp. 6357–6368.
- [11] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *International conference on machine learning*. PMLR, 2021, pp. 12 878–12 889.
- [12] X. Shi, W. Zhang, M. Wu, G. Liu, Z. Wen, S. He, T. Shah, and R. Ranjan, "Dataset distillation-based hybrid federated learning on non-iid data," *arXiv preprint arXiv:2409.17517*, 2024.